

UNIVERSITÉ DE TECHNOLOGIE DE BELFORT-MONTBÉLIARD

SY43, Android Development



UTBM Connect

A Unified Mobile Companion for UTBM Students

PROJECT REPORT

BY **Blusk ATSIZ, Mohamed EL AZRAK, Farès GRIEF and Mawunyo ETOH**

SPRING 2026

Table of Contents

1. Introduction.....	3
2. Project Specification.....	3
2.1 Description of the Chosen Application Theme	3
2.2 Target Users and Expected Use Cases	4
2.3 Functional and Non-Functional Requirements	4
3. Conceptualization of the Application	6
3.1 System Architecture and Main Components	6
3.2 UML Diagrams.....	6
3.3 Database Schema.....	9
4. Implementation Details	10
4.1 Technologies and Tools Used.....	10
4.2 Key Functionalities and How They Were Implemented.....	11
4.3 Main Challenges Encountered	13
5. Screenshots and User Interface	13
5.1 Authentication and Onboarding	13
5.2 Profile Management	15
5.3 Home Feed and Post Creation	16
5.4 Timetable and OCR Import	18
5.5 Messaging	21
5.6 Peer Collaboration	22
5.7 Campus Dining	23
5.8 Administration and Settings.....	24
6. Testing and Results.....	26
6.1 Test Scenarios and How the Application Was Validated	26
6.2 Bugs Encountered and How They Were Resolved.....	27
7. Conclusion and Future Improvements	28
7.1 Summary of the Overall Experience.....	28
7.2 Potential Enhancements and Future Developments	28

1. Introduction

UTBM Connect is a native Android app built to cover the everyday digital needs of a UTBM student in one place: the timetable, an academic news feed, messaging with classmates and teachers, peer revision sessions, and the menu at the university restaurant (RU). The idea behind it is simple. Instead of juggling paper schedules, several messaging apps and a handful of separate institutional tools, a student opens one app and finds everything there.

This report walks through how the application was specified, designed, built and tested. We start with the project specification, then the conceptual design (architecture, UML diagrams, database schema), then the implementation choices and the challenges we ran into along the way. After that comes a tour of the final interface through screenshots, the testing we carried out, and finally a look back at the experience together with ideas for what could still be improved.

2. Project Specification

2.1. Description of the Chosen Application Theme

We designed UTBM Connect as a kind of Swiss-army knife for student life rather than a single-purpose tool. The app covers six areas of everyday student life, all resting on one shared account and one local database:

- **Academic news feed:** a Twitter-style stream of posts (text, images, voice notes or polls) that can be filtered or tagged by course unit (UE) or branch.
- **Timetable / Agenda:** a personal weekly schedule that a student can fill in manually, build from a photo of a printed timetable, or import by pasting the raw text exported from UTBM's planning tool. Professors and administrators can also publish a session directly into students' timetables, either for the whole school or limited to a chosen UE or academic branch.
- **Messaging:** direct conversations, group chats and per-course hubs created by professors, with voice messages, image sharing and official announcements.
- **Peer collaboration:** a board of revision sessions that students can create or join, filtered by campus (Sevenans, Belfort, Montbéliard).
- **Campus dining:** live RU menus pulled from the CROUS open-data API, with automatic detection of the closest restaurant via GPS.
- **Community moderation:** an administrator role that can manage user accounts and moderate the feed.

The idea is that a first-year or mid-curriculum student should be able to open one app in the morning, see their courses, catch up on class announcements, find a revision partner and check the RU menu, instead of switching between five different tools.

2.2. Target Users and Expected Use Cases

Actor	Description	Typical use cases
Student	Default role STUDENT for any UTBM student who signs up.	Sign up / sign in. Pick a branch and up to 10 UEs. Read and post on the feed. Manage the personal timetable. Chat with peers. Create or join a revision session. Check the RU menu.
Professor	Account with the PROFESSOR role, used by teaching staff.	Create a course hub (group conversation) for a UE. Send official announcements that are visually flagged to students. Post course-related content on the feed. Publish a timetable event that is visible to every student, or limit it to specific UEs or branches.
Administrator	A student account with the isAdmin flag turned on.	Open the dedicated Admin tab in the bottom navigation. List all registered users. Delete an account that violates the rules of the platform. Publish a timetable event that is visible to every student, or limit it to specific UEs or branches.

2.3. Functional and Non-Functional Requirements

2.3.1. Functional requirements

Domain	Requirement
Authentication	The system lets a user create an account (name, e-mail, password) and sign in. Passwords are hashed (SHA-256) before being stored. Each account has a role (STUDENT / PROFESSOR) and an optional administrator flag.
Onboarding	On first connection, the system asks the student to choose an academic branch and up to 10 UEs relevant to that branch. The 11th selection is blocked, with explicit feedback shown to the student.
News feed	The system lets any user publish a post containing text, an image, a voice note, or a poll, optionally tagged with a UE or a branch. Other users can comment on a post. Polls let users vote and show live vote counts.

Domain	Requirement
Timetable	The system lets a user view a personal weekly timetable, add events manually, scan a printed timetable with the camera (OCR), or paste a raw text export. Recurring weekly and bi-weekly (group A / group B) sessions are expanded automatically across the semester. A professor or an administrator can also publish an event directly from the Agenda screen, either making it visible to every student or restricting it to one or more target UEs and/or academic branches; the event then appears automatically in the personal timetable of every student who matches that target.
Messaging	The system supports one-to-one conversations, group conversations and per-course hubs. Professors can flag a message as an official announcement. Text, image and voice messages are all supported.
Collaboration	The system lets a user create a revision work session (UE, campus, room, time slot, maximum number of participants) and lets other users join it, filtered by campus. A session can be shared through the native Android share sheet.
Campus dining	The system displays the RU menu of a restaurant chosen by the user, or automatically selects the nearest restaurant from the device's GPS position.
Administration	The system lets an administrator list all registered users and delete a given account.
Settings	The system lets a user switch between light and dark theme and log out of their account.

2.3.2. Non-functional requirements

- **Offline-first persistence:** all user-generated content (accounts, posts, messages, timetable, sessions) lives locally in Room/SQLite, so the app stays usable without a permanent network connection. Only the RU menu needs connectivity.
- **Usability and consistency:** the interface follows Material 3 guidelines (Jetpack Compose), with one visual identity (UTBM blue) and support for both light and dark themes.
- **Backward-compatible data model:** the local database has gone through 11 versioned Room migrations without ever forcing a destructive reinstall, so existing accounts and data survive an app update.
- **Basic security:** passwords are never stored in clear text (SHA-256 hashing); sensitive actions, such as deleting a user, are restricted to the administrator role.
- **Device compatibility:** the app targets Android 8.0 (API 26) and above, up to API 36, which covers the large majority of active Android devices.

- **Maintainability:** the codebase follows a layered MVVM architecture with one ViewModel and one Repository per feature, which keeps each screen independent and easier to extend.
- **Resilience:** features that depend on an external resource (camera OCR accuracy, the CROUS API, GPS availability) fall back to a manual alternative instead of failing outright.

3. Conceptualization of the Application

3.1. System Architecture and Main Components

UTBM Connect follows a layered MVVM (Model-View-ViewModel) architecture, the pattern generally recommended for Jetpack Compose apps. A single Activity (MainActivity) hosts a NavigationCompose graph; every screen is a stateless composable function that reads its state from a dedicated ViewModel and sends user events back to it. Each ViewModel exposes Kotlin Flow / StateFlow streams and delegates all data access to a Repository, which acts as the single source of truth for its feature and decides whether to read from the local Room database or from a remote API.

The data layer has three independent parts:

- **Room (SQLite)** is the primary, offline-first store. A single AppDatabase, created once in `ProjetApplication` and exposed as a singleton, currently defines 14 entities and 14 DAOs and has gone through 11 versioned migrations. We also use Firebase to keep a copy of the data online.
- **Retrofit**, paired with `OkHttp` and `kotlinx.serialization`, is used for the one feature that genuinely needs a remote source: fetching RU menus from the CROUS open-data API.
- **A small set of device-sensor wrappers** (CameraX with ML Kit OCR, and the location APIs) feed recognised text or coordinates back up to the ViewModel layer rather than being accessed directly from the UI.

3.2. UML Diagrams

3.2.1. Use Case Diagram

The use case diagram below summarises the functional scope of the app from the point of view of its two main actors at the time it was drawn. The Student actor can sign up (which includes signing in) and, once authenticated, reach the home feed. From there, six optional extensions cover messaging, posting, the collaboration tab, the restaurant menu, the settings, and the timetable. The Administrator actor inherits every student capability and additionally has direct access to moderation actions: deleting a post from the feed, deleting a user, or resetting a user's password.

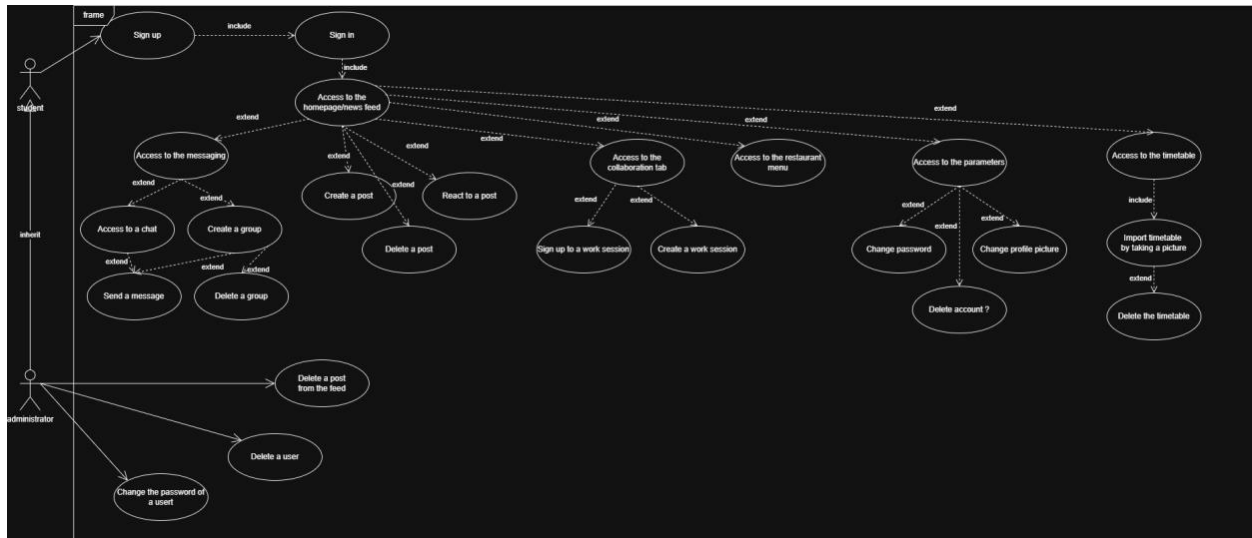


Figure 1. Use case diagram of UTBM Connect (Student and Administrator actors).

A few second-level extensions are detailed further: messaging extends into opening a one-to-one chat or creating a group, which themselves extend into sending a message or deleting a group. The collaboration tab extends into joining or creating a work session. The timetable extends into importing a schedule by taking a picture, which can be followed by deleting the imported timetable. The parameters use case extends into changing the password, the profile picture, or deleting the account.

Figure 1 was drawn early in the project, when professors only had the course hub described in the actor table and no use case of their own on this diagram. Once the privileged event-creation feature was added to the Agenda screen, redrawing the whole diagram did not seem worth it, so Figure 2 below isolates just the addition. A Professor or an Administrator can publish an agenda event, which extends into three options: making it visible to everyone, targeting it to one or more UEs, or targeting it to one or more academic branches. This use case is itself an extension of “Access to the timetable” from Figure 1.

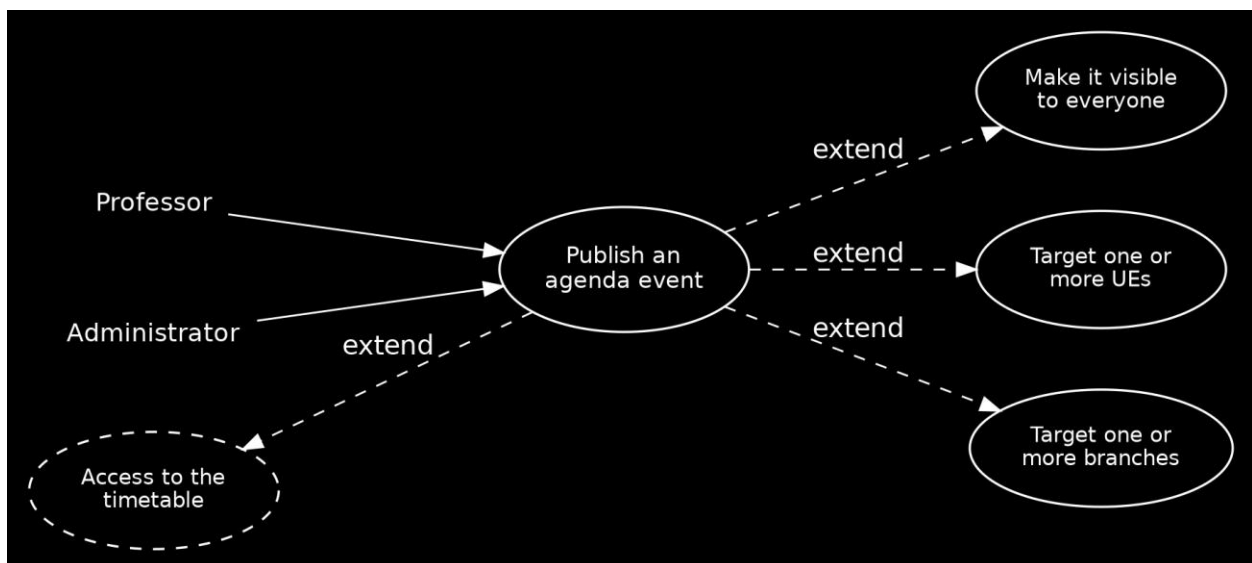


Figure 2. Addition to the use case diagram: publishing an agenda event (Professor and Administrator).

3.2.2. Activity Diagram: Timetable Import via OCR

Because it combines a hardware sensor, a machine-learning component and a custom parsing algorithm, the timetable-import-by-camera use case felt like the most representative flow to detail as an activity diagram. It shows how the app degrades gracefully at each failure point, such as a permission refusal or an unreadable photo, instead of crashing.

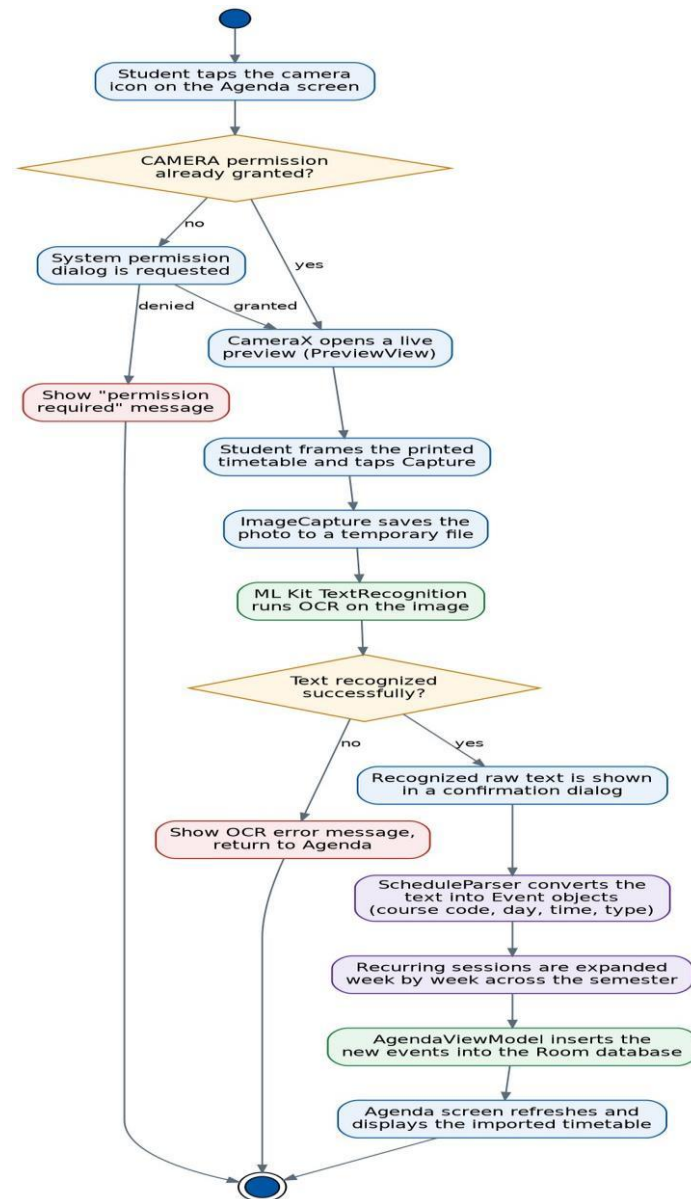


Figure 3. Activity diagram of the "import timetable by taking a picture" use case.

Once the camera permission is granted and a photo is taken, Google ML Kit's on-device text recogniser extracts raw text from the image. That text is shown to the student for a quick check, then handed to the custom ScheduleParser, which recognises course codes, session types (CM/TD/TP/EXAM), days, time slots

and weekly frequency, and expands every recurring session into one concrete Event per week across the current semester before inserting them into Room through the AgendaViewModel.

3.3. Database Schema

The local database is built with Room and currently defines 14 entities. User accounts, UEs and the User-UE enrolment table form the academic backbone. Post, Comment, Poll, PollOption, PollVote and VoiceMessage support the news feed. ChatItem and Message support messaging. Event supports the timetable. Collaboration and CourseMaterial support, respectively, peer revision sessions and course resources. Most relationships are simple one-to-many associations enforced with Room foreign keys and ON DELETE CASCADE, so deleting a user or a post automatically cleans up its dependent rows.

The Event entity is worth a closer look since it is also the one behind the publishing feature described earlier: besides the title, code, location and time fields, it stores an instructor name, an isGlobal flag, and two optional lists, targetUECodes and targetBranches. A student's own manual or OCR-imported sessions are simply saved with isGlobal left at its default value of true, while a session published by a professor or an administrator through the Agenda screen's creation dialog can turn isGlobal off and fill in one or both lists instead. AgendaViewModel reads these fields for every student to decide which events belong in their personal timetable.

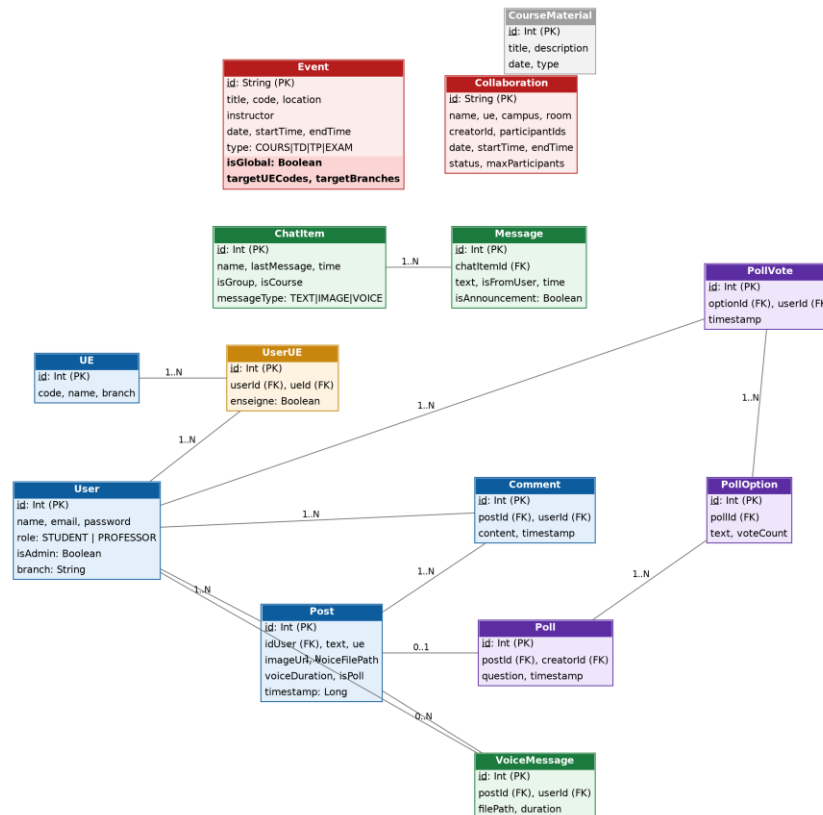


Figure 4. Entity-relationship diagram of the Room database (final schema, version 11). The two highlighted rows in the Event entity, isGlobal and the target lists, are what drives the visibility of a published agenda event.

Entity	Purpose
User	A registered account (student, professor or administrator) with a hashed password and an academic branch.
UE	Catalogue of UTBM course-unit codes available for selection (seeded with about fifty real UE codes).
UserUE	Many-to-many enrolment between a User and the UEs they follow or teach (capped at 10 per student).
Post	One feed item: text, an optional image, an optional voice note, an optional poll, optionally tagged with a UE.
Comment	A reply attached to a Post.
Poll / PollOption / PollVote	A poll attached to a Post, its selectable answers, and the votes cast by users.
VoiceMessage	Metadata (file path, duration) of a recorded voice note attached to a Post.
ChatItem	A conversation thread: a direct message, a group, or a course hub.
Message	One message inside a ChatItem, optionally flagged as an official announcement.
Event	One timetable entry (course, TD, TP or exam). An isGlobal flag plus the optional targetUECodes and targetBranches lists decide whether the entry reaches every student or only a chosen UE or branch, which is what lets a professor or an administrator broadcast a session instead of just keeping it private.
Collaboration	A peer revision work session: UE, campus, room, time slot, participants and capacity.
CourseMaterial	A document or resource shared inside a course hub.

4. Implementation Details

4.1. Technologies and Tools Used

Layer	Technology	Role in the project
Language	Kotlin	Primary application language.
UI toolkit	Jetpack Compose + Material 3	Declarative UI, theming, navigation bar, dark mode.
Navigation	Navigation-Compose	Single-Activity navigation graph between screens.
Local persistence	Room (SQLite) + KSP	14 entities / 14 DAOs, 11 versioned migrations, offline-first storage.

Layer	Technology	Role in the project
Networking	Retrofit2 + OkHttp + kotlinx.serialization	REST client for the CROUS open-data menu API.
Image loading	Coil	Asynchronous loading and caching of post and profile images.
Camera & OCR	CameraX + Google ML Kit Text Recognition	Live camera preview, photo capture and on-device OCR for timetable import.
Location	Android location permissions + custom Haversine calculation	Detection of the nearest CROUS restaurant from the device's GPS position.
Build system	Gradle (Kotlin DSL) with a version catalog	Dependency management (libs.versions.toml), build variants.
Testing scaffolding	JUnit4 + Espresso	Unit and instrumentation test entry points (see Section 6).

4.2. Key Functionalities and How They Were Implemented

4.2.1. Authentication and onboarding

The Connexion and Register composables (ui/Register) collect a name, a UTBM e-mail address and a password, hash the password with SHA-256 through PasswordUtils, and create or look up a User row through UserRepository / UserViewModel. The sign-up form also has an optional “Code d'accès” field: entering a valid code elevates the new account directly to the Professor role, or to Administrator, instead of the default Student role. A new account is then guided through a one-time WelcomeScreen where it has to choose an academic branch and up to 10 relevant UEs. The selection logic disables further checkboxes once the 10-UE quota is reached, with a counter that turns red as a visual cue.

4.2.2. News feed, posts, comments and polls

HomeScreen and PostViewModel implement the main feed. Creating a post (CreatePostScreen) supports four content types: plain text, an attached image (picked from the device and loaded with Coil), a recorded voice note (stored as a VoiceMessage and played back inline), or a poll (a Poll entity with several PollOption rows). PostRepository centralises insert, delete and count operations, and CommentDao lets any user reply under a post.

4.2.3. Timetable / Agenda

AgendaScreen and AgendaViewModel manage the Event entity. Three input paths converge on the same data for a student's own planning: manual creation, a camera-based OCR scan (CameraScreen + ML Kit, see Figure 3), or pasting a raw ADE export into a dialog. Both automatic paths are parsed by the same ScheduleParser, which recognises the course code, session type (CM / TD / TP / EXAM), day, start and end time, and a weekly or bi-weekly frequency, then iterates week by week across the detected semester

range to generate one Event per actual session, correctly alternating between group A and group B for bi-weekly courses.

Accounts with the Professor or Administrator role see one extra control on this same screen: a floating action button that opens a dedicated creation dialog (Figure 15, Section 5.4). The dialog asks for the usual event details, title, UE code, room, instructor, date, start and end time, type, plus a visibility setting. A switch labelled “Visible par tous” defaults to on; turning it off reveals two chip pickers, one for target UEs and one for target academic branches, so the event can be aimed at a specific population instead of the whole school. AgendaViewModel keeps a small isVisibleForUser check that combines the event's isGlobal flag with the signed-in student's enrolled UEs and branch: a global event always shows, and a targeted event shows only if the student is enrolled in one of the listed UEs or belongs to one of the listed branches. Because the same Event table and the same filtering logic are shared with the personal-import paths described above, a published event simply appears in the right students' weekly view the next time their Agenda screen refreshes, with no separate notification mechanism needed for now.

4.2.4. Messaging

ChatScreen, ConversationScreen and CourseScreen, backed by ChatViewModel and ChatRepository, manage three kinds of ChatItem: direct conversations, groups, and professor-created course hubs. Messages carry a type (text, image or voice) and an isAnnouncement flag that professors can set so the message stands out visually as an official communication inside a course hub.

4.2.5. Peer collaboration (revision sessions)

The Collaboration entity, exposed through CollaborationViewModel and CollaborationScreen, models a work session with a campus, a room, a time slot and a maximum number of participants. Students can filter the board by campus, create a session, or join an existing one. The screen also lets a session be shared through the native Android Intent share sheet, for example to forward it to a class group chat.

4.2.6. Campus dining (RU menu)

RestaurantViewModel wraps a Retrofit CrousApiService pointed at the CROUS open-data API (api.croustillant.menu). It first tries to fetch today's menu for the selected restaurant. If that request fails or comes back empty, it falls back to the full list of available menu days for that restaurant, and if that also fails, it shows an explanatory message instead of a raw error. The nearest restaurant among the four seeded campuses can also be auto-selected from the device's GPS coordinates using a Haversine great-circle distance calculation.

4.2.7. Administration

AdminScreen and AdminViewModel are only reachable from the bottom navigation bar when the signed-in account has isAdmin = true. The screen lists every registered user with their current role and offers

three actions per account: promote a Student to Professor, grant or revoke the Administrator right, and permanently delete the account.

4.2.8. Settings and theming

SettingsScreen lets the user toggle between light and dark themes, with the system theme used as the default on first launch, and log out, which resets the in-memory session state and returns to the sign-in screen.

4.2.9. Motion and feedback animations

A few targeted animations were layered on top of the static screens described above, mostly to make state changes easier to follow rather than to decorate the interface. On the Home feed, each post now fades in and slides up slightly when the feed first loads or refreshes, with a short staggered delay between posts (about 70 ms per post, capped at 280 ms), so the list reveals itself as a quick cascade instead of popping in all at once; this relies on Compose's `AnimatedVisibility` combined with `fadeIn` and `slideInVertically`. The like button on a post reacts the same way most social apps do: tapping the heart scales it up with a bouncy spring animation and switches its tint from white to red, while the like count next to it slides and fades between values through `AnimatedContent` instead of jumping straight to the new number. On the Chat tab, conversations are now keyed by their id in the `LazyColumn` and wrapped in `Modifier.animateItem(tween(300))`, so when a conversation moves up the list after a new message arrives, it slides smoothly into its new position instead of jumping there instantly.

4.3. Main Challenges Encountered

- **Evolving the data model safely.** The schema went through 11 successive Room migrations instead of one final design settled up front. Migration 3→4, for instance, rebuilt the User table to introduce an explicit role column instead of overloading the `isAdmin` flag, and migration 7→8 introduced the UE / UserUE many-to-many relationship. Every migration had to be written as additive SQL (`CREATE TABLE` / `ALTER TABLE`) so accounts and content created earlier in development would survive instead of being wiped.
- **Unreliable third-party data.** The CROUS API regularly returns a 404 response for restaurants that have not published a detailed menu for the current day, which we ran into often with the Belfort campus. We handled this with a three-step fallback chain, today's menu, then the full list, then a clear explanatory message, instead of letting the screen show a raw network error.
- **OCR reliability.** Recognising a printed or screen-displayed timetable with ML Kit is sensitive to lighting, glare and table layout, so results are not always usable on the first try. We added a manual “paste raw text” alternative, parsed by the exact same `ScheduleParser`, so the feature never becomes unusable when a photo fails to produce clean text.

- **Runtime permissions.** Camera access and location are runtime permissions on modern Android, so the corresponding flows had to handle a refusal gracefully, with an explanatory state plus a retry button, instead of crashing.
- **Recurring schedule generation.** Expanding a single scanned week into the right set of weekly and bi-weekly (group A / group B) sessions across a whole semester, without duplicating or skipping a week, needed dedicated date-iteration logic based on the detected semester boundaries rather than a fixed week count.
- **Designing event visibility without overcomplicating the model.** Once professors and administrators could publish events too, we needed a way to say “this is for everyone” or “this is only for these UEs / branches” without adding a second table. We settled on a single `isGlobal` flag plus two optional lists on the existing Event entity, and a small `isVisibleForUser` check in `AgendaViewModel` that every student's event stream is filtered through. The creation dialog also keeps the “Créer” button disabled until either the visibility switch is on or at least one UE or branch has been picked, so it is not possible to publish an event that nobody would ever see.

5. Screenshots and User Interface

This section walks through the main screens of the finished app and how a user moves between them. Most of the screenshots below were taken directly from the running application.

5.1. Authentication and Onboarding

The sign-in screen shows the UTBM Connect logo and asks for a UTBM e-mail address and password. The “Se connecter” button stays disabled until both fields are filled, and a link below leads to the sign-up screen for new accounts.



The image shows a mobile application sign-in screen for UTBM Connect. At the top, there is a logo consisting of a blue square with a white 'U' and the text 'UTBM connect' to its right. Below the logo, the text 'Connectez-vous à votre compte' is displayed. The main form area contains two input fields: the first is labeled 'Email (@utbm.fr)' with an envelope icon, and the second is labeled 'Mot de passe' with a lock icon. Below these fields is a grey button with the text 'Se connecter'. At the bottom of the form, there is a link that says 'Pas encore de compte ? S'inscrire'.

Figure 5. Sign-in screen.

The sign-up screen collects a name, a UTBM e-mail address, a password and an optional access code, along with a role badge. By default the new account is a Student. Entering a valid access code immediately switches the badge below the form to “Étudiant · Administrateur” (or “Professeur”), showing which elevated role will be granted.

The sign-up screen features the UTBM connect logo at the top with the text "Créer votre compte". Below the logo are four input fields: "Nom" (with a person icon), "Email (@utbm.fr)" (with an envelope icon), "Mot de passe" (with a lock icon), and "Code d'accès (optionnel)" (with a shield icon). A light blue button labeled "Étudiant" is positioned below the input fields. At the bottom, there is a large grey "S'inscrire" button and a link "Déjà un compte ? [Se connecter](#)".

Figure 6. Sign-up screen (default Student role).

This screen is identical to Figure 6 but with the "Étudiant" button highlighted in orange and labeled "Étudiant · Administrateur". Additionally, a hamburger menu icon is visible in the top right corner. The "S'inscrire" button is now blue, and the link "Déjà un compte ? [Se connecter](#)" remains at the bottom.

Figure 7. After entering a valid access code (Administrator role).

Right after a first sign-up, the Welcome screen greets the student by name and asks them to pick an academic branch. Once a branch is picked, the matching list of UEs appears below so the student can select up to 10 of them, with a counter and a “Commencer” button to confirm.

5.2. Profile Management

This branch and UE selection can also be revisited later from the profile screen, reached from the person icon in the Home header, which also shows the account's name, e-mail and role, with actions to edit the profile or change the password. Each enrolled UE can be removed individually with the red cross, and the counter, 6/10 here, keeps the student from going over the ten-UE limit.

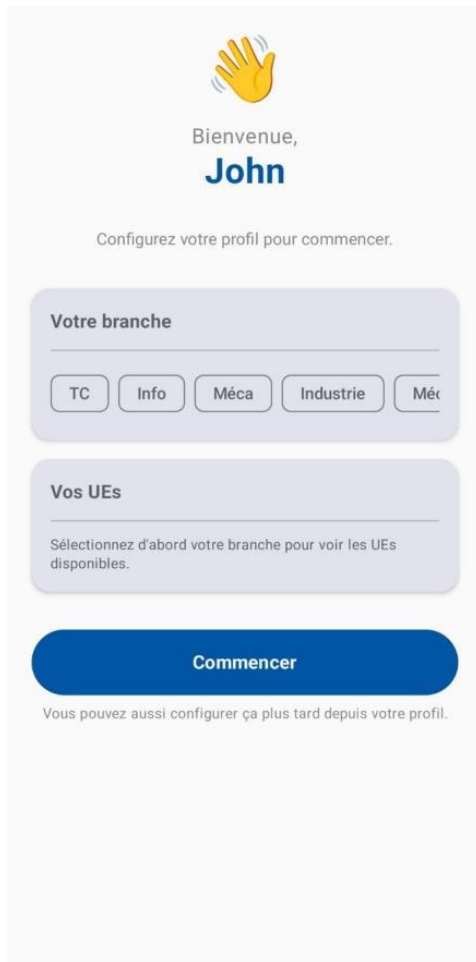


Figure 8. Welcome / onboarding screen (branch selection).

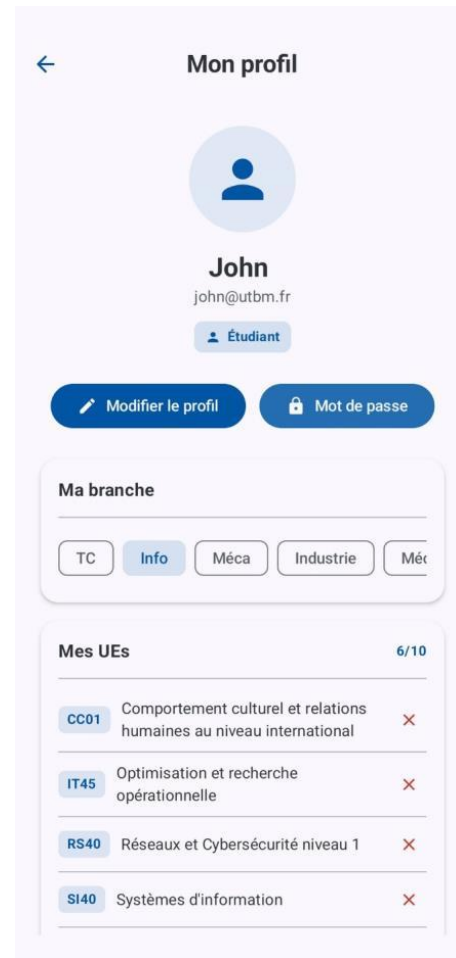


Figure 9. Profile screen.

5.3. Home Feed and Post Creation

The Home tab greets the student by name ("Bonjour, John") and shows the day's feed, with quick filter chips to narrow it down to a specific UE (Général, Info, CC01, IT45...). Each post can be expanded to reveal the like, comment and share actions, as shown below. The floating action button (+) opens the post-creation screen, where the user picks a content type, text, image, voice note or poll, before publishing.



Figure 10. Home feed screen, with a post expanded to show its reactions.

The post-creation screen lets the author pick which UE the post belongs to (or “Général” for the whole school) and which of three content types to publish: a plain text post, a voice note recorded directly in the app, or a poll with at least two options.

← Créer un post

Publier dans

Général Info CC01 IT45 RS4

Texte Vocal Sondage

Quoi de neuf ?

Insérer une image

Publier

Figure 11. Text mode.

← Créer un post

Publier dans

Général Info CC01 IT45 RS4

Texte Vocal Sondage

Enregistrer un message vocal

Appuyez pour enregistrer

Publier

Figure 12. Voice-note mode.

← Créer un post

Publier dans

Général Info CC01 IT45 RS4

Texte Vocal Sondage

Question du sondage

Options

Option 1

Option 2

+ Option

Publier

Figure 13. Poll mode.

5.4. Timetable and OCR Import

The Agenda tab opens on an import panel (“Ajouter votre planning”) offering two ways to fill a personal timetable, a camera/OCR scan or pasting raw ADE text, above the weekly planner (“Mon Planning”) with day-by-day navigation. The screenshot below shows the result of an import: each session is colour-coded by type (grey for “Cours”, yellow for “TD”), tagged “Importé”, and the day strip shows a dot under every day that has at least one session.

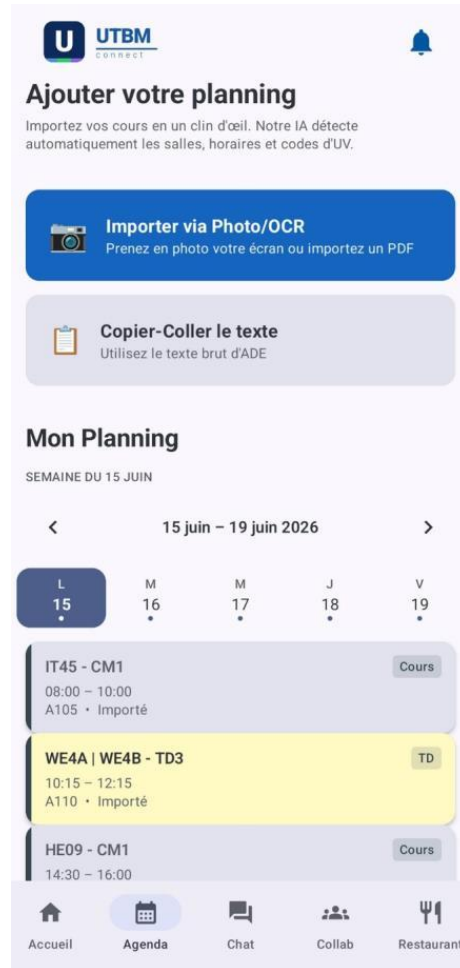


Figure 14. Agenda screen after a timetable import (IT45, WE4A/WE4B, HE09).

On a Professor or Administrator account, the same screen also shows a floating “+” button in the bottom-right corner, which a Student account never sees. Tapping it opens the dialog in Figure 15, where the title, UE code, room, instructor, date and time are filled in like any other event, and the “Visible par tous” switch decides whether the session reaches every student or only the UEs and branches picked from the chip pickers below it. The example shown here targets the SY43 UE and the Info branch specifically, rather than the whole school.

Ajouter votre planning
Importez vos cours en un clin d'œil.

Nouvel événement

SY43 - Soutenance de projet

SY43

Amphi Nord

M. Lefèvre

2026-06-25

14:00 16:00

Visible par tous ☐

UEs concernées

SY43×

+ Ajouter une UE

Branches concernées

Info×

+ Ajouter une branche

Cours TD TP Exam

Annuler Créer

Accueil Agenda Chat Collab Restaurant

Figure 15. Event-creation dialog reserved for Professor and Administrator accounts, here targeting UE SY43 and branch Info instead of broadcasting to everyone.

“Copier-Coller le texte” opens a simpler dialog where the student pastes the raw text exported from ADE instead. The “Ajouter” button stays disabled until the text field is filled.



Figure 16. Paste-text dialog.

5.5. Messaging

The Chat tab provides quick filters (Tout / Non lus / Groupes / Cours), a highlighted course-hub card for official announcements, and the list of conversations with the last message, a timestamp, an unread indicator and a distinct icon for voice messages. Opening a conversation shows the full message history as chat bubbles, the sender's messages on the left and the user's own on the right, each with its timestamp, plus a text field to reply. Voice messages and professor announcements reuse the same bubble layout with a distinct icon or highlight colour.

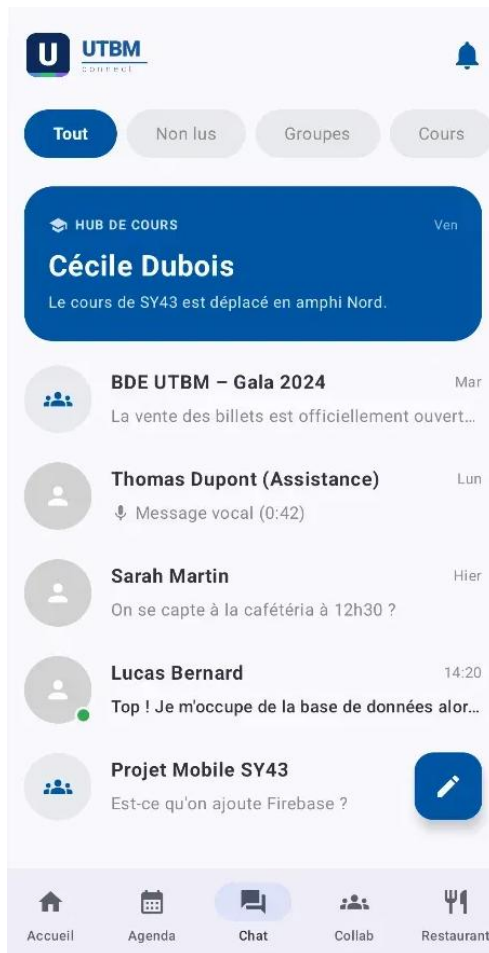


Figure 17. Chat list screen.



Figure 18. Conversation screen.

5.6. Peer Collaboration

The Collab tab lists open revision sessions tagged by UE, with campus filter chips, the date, time and room, the current number of participants out of the session's capacity, a share icon, and a “Rejoindre” button that turns into a disabled “Complet” label once the session is full. Tapping “+ Créer” opens the session-creation form: a name, the UE code, an optional description, the campus, the room, the date and the start / end time.

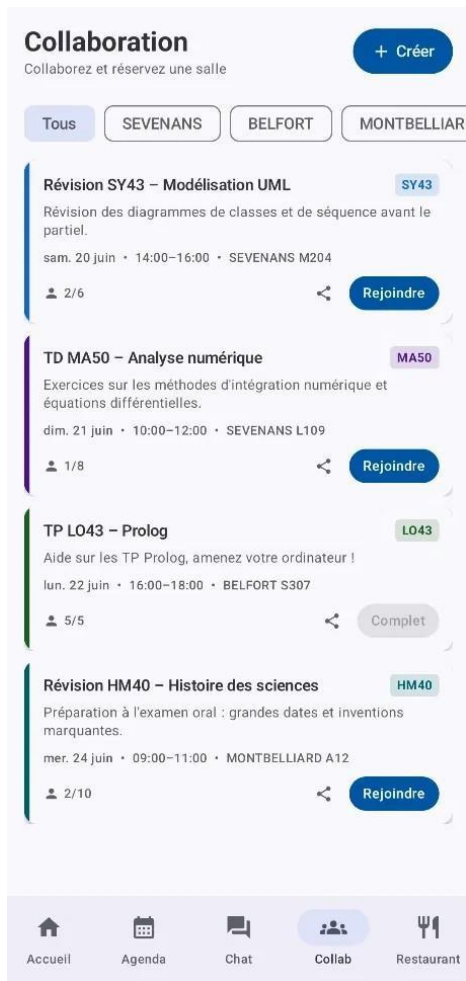


Figure 19. Collaboration board screen.

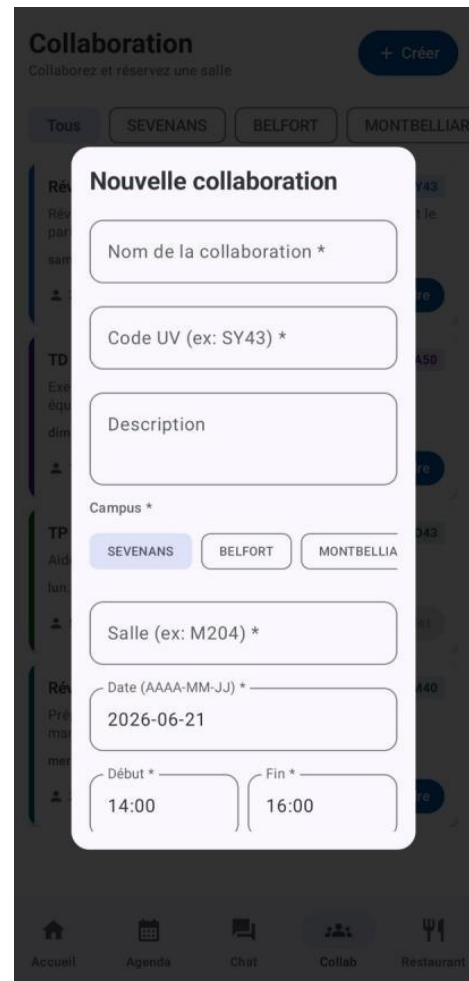


Figure 20. Create work session dialog.

5.7. Campus Dining

The Restaurant tab lets the student switch between the four campuses, shows the selected restaurant and the menu date, and lists each dish grouped by category, here “Laitage et fruits” for the midday service, with a GPS button to auto-detect the closest restaurant and a refresh button to reload the menu.



Figure 21. RU menu screen (Sevenans campus).

5.8. Administration and Settings

Visible only to administrator accounts, the Admin tab (“Gestion des utilisateurs”) lists every registered user with their current role badge. Each card offers two promotion toggles, “→ Professeur” and “+ Admin” / “Retirer admin”, alongside a delete action. The seeded “Admin Demo” account cannot be demoted from its own card by mistake, since “Retirer admin” is shown instead of a second promotion option.

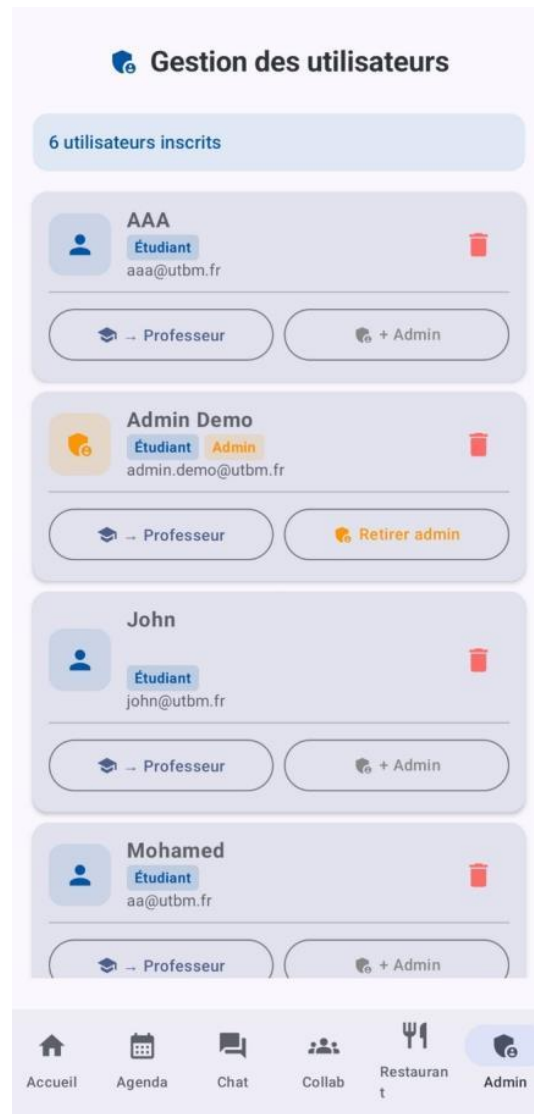


Figure 22. Admin user-management screen.

The Settings screen exposes the dark-mode toggle, a push-notifications switch, the app version, and the logout action. The screenshot below was taken with dark mode enabled, which carries over to the rest of the app, for example the Home feed shown right next to it.

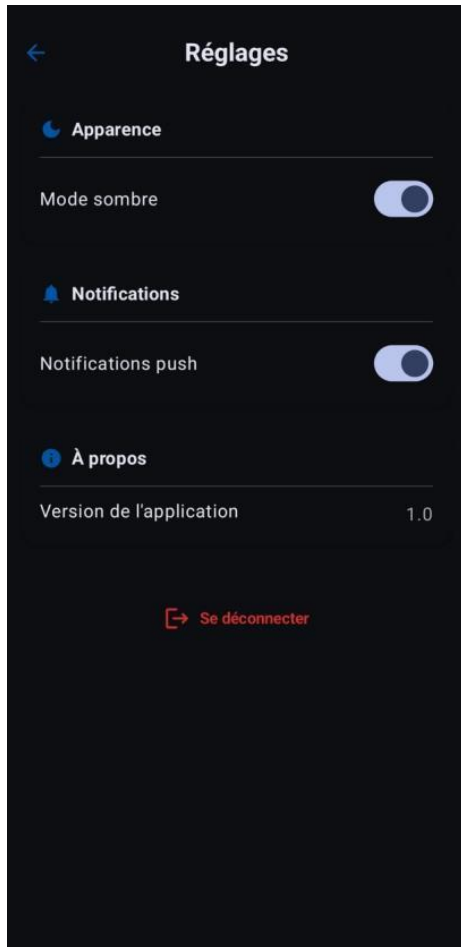


Figure 23. Settings screen (dark mode).



Figure 24. Home feed screen in dark mode.

6. Testing and Results

Testing was carried out manually throughout development, screen by screen, since most of the project's time budget, proposal in March and final delivery in June, went into the breadth of features described above. The JUnit / Espresso scaffolding generated by Android Studio (ExampleUnitTest, ExampleInstrumentedTest) was kept as a starting point for automated tests rather than expanded into a full suite (see Section 7.2). The table below lists the main scenarios we exercised; exact dates and devices used during the real test sessions should still be filled in by the group.

6.1. Test Scenarios and How the Application Was Validated

Scenario	Expected result / observed behaviour
Sign up with a new e-mail address	Account is created and hashed in the User table; the app moves to the Welcome onboarding screen.

Scenario	Expected result / observed behaviour
Pick more than 10 UEs	The 11th checkbox is disabled and the UE counter turns red, exactly as implemented in WelcomeScreen.
Create a text, image, voice or poll post	The new post appears at the top of the Home feed immediately, with the correct content type badge.
Vote on a poll	The selected option's vote count increases and is visible to every user viewing that post.
Import a timetable by camera	The recognised text is shown for confirmation, then the corresponding Event rows appear on the right day; quality depends on photo conditions (see Section 4.3).
Import a timetable by pasting ADE text	All CM / TD / TP / EXAM lines are converted into events on the correct weeks, including the group A / B bi-weekly alternation.
Send a message in a direct conversation	The message appears instantly at the bottom of the thread, ordered by time.
Professor sends an announcement in a course hub	The message is visually flagged as official to every member of that hub.
Professor or Administrator publishes an event for everyone, a target UE, or a target branch	The event is created with the matching isGlobal / targetUECodes / targetBranches values; it then shows up in the personal Agenda of every student who is global-eligible, enrolled in the chosen UE, or belonging to the chosen branch, and stays out of every other student's view.
Create a work session and fill it to capacity	A new participant cannot join once maxParticipants is reached.
Fetch the RU menu with and without GPS	With GPS, the nearest of the four seeded restaurants is auto-selected; without it, the user can pick one manually.
CROUS API returns no data for the current day	The app falls back to the full list, then to an explanatory message, instead of crashing.
Administrator deletes a user	The account disappears from the Admin list and can no longer sign in.
Toggle dark mode	The whole interface switches theme immediately and keeps the system default on first launch.

6.2. Bugs Encountered and How They Were Resolved

Issue observed	Resolution
The User table initially conflated the “is teacher” and “is administrator” notions in a single isAdmin flag, which made permission checks inconsistent.	The table was migrated (3→4) to a clean schema with both an explicit role column (STUDENT / PROFESSOR) and a separate isAdmin flag.

Issue observed	Resolution
The CROUS API returns HTTP 404 for several restaurants (e.g. Belfort) when no detailed menu has been published yet for the day.	Added the three-step fallback described in Section 4.3 (today → full list → explanatory message) instead of surfacing a raw error to the user.
Photos of a printed or screen-displayed timetable sometimes produce text ML Kit cannot parse (glare, low contrast, skewed angle).	Kept the camera path as the primary, fast option but added a manual paste fallback using the same parser, so the feature degrades instead of failing.
A naive “one event per detected line” import duplicated or skipped sessions for bi-weekly group A / B courses across a full semester.	Rewrote ScheduleParser to iterate week by week from the detected semester start, applying an explicit odd/even week check for each group.
Relying only on fallbackToDestructiveMigration() during early development risked silently wiping test accounts and posts on every schema change.	Added an explicit, additive Migration object for every schema version and kept the destructive fallback only as a last-resort safety net.
A professor or administrator could turn off “Visible par tous” in the event dialog without picking any UE or branch, which would have created an event that no student could ever see.	Added a targetsValid check in CreateEventDialog that keeps the “Créer” button disabled until the switch is back on or at least one UE or branch has been added.

7. Conclusion and Future Improvements

7.1. Summary of the Overall Experience

Building UTBM Connect over the course of the SY43 module gave the group hands-on experience with the modern Android stack: Kotlin and Jetpack Compose for the UI, an MVVM architecture with Repositories and Flow-based state, Room for local persistence and schema migration, Retrofit for consuming a real external API, and CameraX with ML Kit for a sensor-driven feature. Coordinating four contributors around a single shared data model was itself a real exercise: the eleven successive Room migrations trace the project's growth, from a simple authentication flow at the proposal stage in March 2026 to a six-feature application by the June 2026 deadline.

The main trade-off of the project was breadth versus polish. Covering six fairly different feature areas, feed, timetable, messaging, collaboration, dining, administration, left less time for automated testing and UI refinement than a narrower, single-feature app would have allowed. Section 6 reflects that directly, since validation stayed largely manual.

7.2. Potential Enhancements and Future Developments

- Improve the Firebase side with better data organisation.
- Replace the camera-OCR / paste timetable import with a direct integration to UTBM's ADE export or API, removing the need for any manual step.

- Add push notifications for new messages, professor announcements, poll results, and newly published agenda events.
- Build a real automated test suite (unit tests for ScheduleParser and the Repositories, UI tests for the main screens) on top of the existing JUnit / Espresso scaffolding.
- Add end-to-end encryption for one-to-one conversations.
- Run an accessibility pass (TalkBack support, dynamic font scaling, colour-contrast check) on every screen.
- Add multi-language support (French / English) to better serve UTBM's international student population.